

Chunkwise KLA with Dense Boundary Covariance: A TTT-Style Approximation and Implementation Plan

August 20, 2026

1 Decision and Scope

The previous plan attempted to reproduce tokenwise ExactKLA exactly by factoring the joint innovation covariance inside every chunk. That route has now been measured extensively and is not the implementation target of this note. Exact prefix conditioning retains the expensive dynamic cross-covariance builder, a small dense factorization and solve, a wide right-hand side for causal reads and boundary state, and a dense covariance update. Chunking changes the schedule, but not enough of the work.

The replacement is a different model, called *Chunk-KLA* in this note. It keeps a dense covariance between chunks but deliberately approximates inference within a chunk. All tokens in a chunk form gains from the same dense boundary prior; the mean then follows a causal sequential-residual update whose gains remain frozen. This update has the existing fast affine/WY chunk form. The covariance committed at the boundary is dense and is the Joseph-form ensemble error covariance of that approximate estimator. Thus approximate within the chunk does not mean diagonal covariance: the uncertainty carried across chunks remains anisotropic and dense.

This is not a new backend for ExactKLA. It must be implemented and evaluated under a distinct mixer name such as `chunk_kla` or `ttt_kla`. The recurrent ExactKLA and the exact direct-conditioning work remain useful inference targets and correctness references, but they are not production implementation plans for this model.

1.1 Reference: tokenwise ExactKLA

For comparison, tokenwise ExactKLA predicts, assimilates one observation, and reads after every token:

$$\begin{aligned}\hat{\mathbf{S}}_t &= \mathbf{D}_t \mathbf{S}_{t-1}, & \hat{\mathbf{P}}_t &= \mathbf{D}_t \mathbf{P}_{t-1} \mathbf{D}_t^\top + \boldsymbol{\Omega}_t, \\ \boldsymbol{\kappa}_t &= \frac{\hat{\mathbf{P}}_t \mathbf{k}_t}{r_t + \mathbf{k}_t^\top \hat{\mathbf{P}}_t \mathbf{k}_t}, & \mathbf{S}_t &= \hat{\mathbf{S}}_t + \boldsymbol{\kappa}_t (\mathbf{v}_t - \hat{\mathbf{S}}_t^\top \mathbf{k}_t)^\top, \\ \mathbf{P}_t &= (\mathbf{I} - \boldsymbol{\kappa}_t \mathbf{k}_t^\top) \hat{\mathbf{P}}_t, & \mathbf{o}_t &= \mathbf{S}_t^\top \mathbf{q}_t.\end{aligned}\tag{1}$$

Here $\mathbf{D}_t = \text{diag}(\boldsymbol{\alpha}_t)$, $\boldsymbol{\Omega}_t = \text{diag}(\boldsymbol{\omega}_t)$, and the posterior covariance is dense. The recurrence costs $O(d_k^2)$ arithmetic and one dense covariance dependency per token.

1.2 Why the exact chunk plan is retired

The measurements in `sanboxes/kernels/exact_kla_log.md` identify the decisive bottlenecks:

Measurement	Consequence
Diagnostics-off $d_k = 128$ screening run: 77.37 ms versus 2.63 ms for KDA ($29.4\times$); non-final 1/4 sample	Exact direct conditioning is not close enough to justify another generic decomposition, but this ratio is not acceptance evidence.
Builder/factor/solve/epilogue: 54.94/7.60/6.98/4.66 ms	The dynamic exact builder is the dominant cost; solver-only optimization cannot rescue the complete layer.
Prepared $C = 32$ two-CTA, X-only factor/inverse plus 288-RHS chain: 1.34 ms	This excludes the builder, O/S/P posts, guards, layer shell, backward, and a true dependent boundary recurrence. It supports only the two-CTA scheduling mechanics.
Solver-only $C = 32$, 128-call current-stream wrapper: 3.54 ms	This includes Python invocation, allocation, bookkeeping, and one launch per call, but no complete chunk recurrence. It shows why that call pattern must not be final.
Optimistic $d_k = 128, C = 128$ TF32 PH diagnostic: approximately 0.66 ms	This is a guards-off composite lower bound, not one promoted configuration or a dependent $C = 32$ recurrence. It motivates measuring a batched dense product; it does not establish the new path’s latency or accuracy.

The new direction therefore tests the reusable pieces—GEMM-shaped PK products, a provisional $C = 32$ tile, and a persistent chunk loop—while deleting exact prefix conditioning, the transition-conditioned J/B builder, Cholesky or LDL^T , and the 288-column exact right-hand side. A different small Gram matrix and boundary-update dataflow remain and must be timed as a complete dependent recurrence. The existing register-resident $C = 32$ exact solver is a benchmark, not a component of the proposed algorithm.

1.3 Layer contract for the first prototype

To keep mixer comparisons meaningful, the first Chunk-KLA layer inherits the same KimiDeltaAttention peripherals as ExactKLA, DiagKLA, and KDA: the $q/k/v$ projections, short convolution, output gate, gated output normalization, and output projection. It uses the same iso-parameter backbone. Queries and keys are L^2 -normalized, and the equations below absorb the usual $d^{-1/2}$ read scale into $\mathbf{q}$.

The parameterization also stays fixed:

$$r_t^{\text{raw}} = r_{\min} + \text{softplus}(r_{\text{proj},t}), \quad \omega_t = q_{\min} + \text{softplus}(\mathbf{q}_{\text{proj},t}), \quad \mu = \text{softplus}(\mu_{\text{param}}) + 0.1. \quad (2)$$

When `dk_calibration=True`, every formula uses the effective noise $r_t = r_t^{\text{raw}}/d$; otherwise it uses $r_t = r_t^{\text{raw}}$. Assume $r_t > 0$, $\mu > 0$, $\alpha_{t,i} > 0$, and $\omega_{t,i} \geq 0$. The initial implementation is a training/prefill path with no cache. Streaming decode is a separate gate because it cannot batch future keys into PK .

2 Proposed Chunk-KLA Model

2.1 Dense boundary prior

Let $d = d_k$, $m = d_v$, and let chunk c contain $C_c \leq C$ real tokens. At its entrance, the model carries

$$\mathbf{S}_c^- \in \mathbb{R}^{d \times m}, \quad \mathbf{P}_c^- \in \mathbb{R}^{d \times d}, \quad \mathbf{P}_c^- \succ 0. \quad (3)$$

The columns of $\mathbf{S}_c^-$ share the same key-space covariance $\mathbf{P}_c^-$, as in KLA. Within the chunk, stack

$$\mathbf{K}_c = [\mathbf{k}_1, \dots, \mathbf{k}_{C_c}] \in \mathbb{R}^{d \times C_c}, \quad \mathbf{Q}_c = [\mathbf{q}_1, \dots, \mathbf{q}_{C_c}] \in \mathbb{R}^{d \times C_c}, \quad (4)$$

and let row t of $\mathbf{V}_c \in \mathbb{R}^{C_c \times m}$ be $\mathbf{v}_t^\top$. Define

$$\mathbf{R}_c = \text{diag}(r_1, \dots, r_{C_c}), \quad r_t > 0. \quad (5)$$

The chunk-time observation model treats the latent associative map as fixed while these observations are accumulated:

$$\mathbf{V}_c = \mathbf{K}_c^\top \mathbf{S}_c^* + \mathbf{E}_c, \quad \text{Cov}(\mathbf{E}_{c,:j}) = \mathbf{R}_c \quad (6)$$

for every value coordinate j . Conditional on the chunk-entry variables and $(\mathbf{K}_c, \mathbf{R}_c)$, the noise is zero mean and independent of the entry-state estimation error. This static-within-chunk assumption is part of the approximation, not a claim about the original tokenwise state-space model.

2.2 Parallel gains from the frozen prior

Form

$$\mathbf{U}_c = \mathbf{P}_c^- \mathbf{K}_c, \quad \mathbf{G}_c = \mathbf{R}_c + \mathbf{K}_c^\top \mathbf{U}_c, \quad (7)$$

$$\mathbf{D}_{g,c} = \text{diag}(\text{diag}(\mathbf{G}_c)), \quad \mathbf{L}_c = \mathbf{U}_c \mathbf{D}_{g,c}^{-1}. \quad (8)$$

Column t of $\mathbf{L}_c$ is

$$\ell_t = \frac{\mathbf{P}_c^- \mathbf{k}_t}{r_t + \mathbf{k}_t^\top \mathbf{P}_c^- \mathbf{k}_t}. \quad (9)$$

It is the scalar-observation Kalman gain for token t , evaluated from the chunk-entry prior. All columns can therefore be computed together with one dense covariance-key product and a diagonal normalization.

For the chunk-static model, the exact joint block gain would be

$$\mathbf{L}_{c,\text{block}} = \mathbf{U}_c \mathbf{G}_c^{-1}. \quad (10)$$

Equation (8) makes the explicit approximation

$$\mathbf{G}_c^{-1} \longrightarrow \mathbf{D}_{g,c}^{-1}. \quad (11)$$

This substitution defines the *raw* per-token gains: each token uses the correct dense boundary covariance but initially ignores innovation correlations with the other tokens. The causal WY transform below then incorporates earlier residual feedback, so the final effective gain is not simply $\mathbf{U}_c \mathbf{D}_{g,c}^{-1}$. There is no innovation-covariance factorization. The mean update uses only the unit-lower solve already present in `chunk_kalman`.

2.3 Frozen-gain sequential residuals and the WY form

First compute the boundary residuals

$$\Delta_c = V_c - K_c^\top S_c^-. \quad (12)$$

The raw gains are frozen, but the mean adapts causally inside the chunk:

$$S_{c,0}^{\text{virt}} = S_c^-, \quad (13)$$

$$\mathbf{e}_t = \mathbf{v}_t - (S_{c,t-1}^{\text{virt}})^\top \mathbf{k}_t, \quad (14)$$

$$S_{c,t}^{\text{virt}} = S_{c,t-1}^{\text{virt}} + \ell_t \mathbf{e}_t^\top. \quad (15)$$

Only S_{c,C_c}^{virt} is committed; the earlier virtual states exist to define causal reads. This is a TTT-inspired minibatch update: the chunk is the minibatch, the memory is the adapted parameter, and the dense boundary covariance preconditions every write. Its approximation is freezing the gain, not freezing the residual.

The recurrence has a compact affine/WY form. Define

$$M_c = \text{tril}(K_c^\top L_c, -1), \quad T_c = I + M_c, \quad \bar{A}_c = T_c^{-1}. \quad (16)$$

Stacking the sequential innovations as rows of E_c gives

$$E_c = \bar{A}_c \Delta_c, \quad S_c^+ = S_c^- + L_c E_c. \quad (17)$$

The unit-lower solve is not the rejected exact Cholesky path: it contains no dynamic state cross-covariance and is the same WY transform used by the existing fast memory kernel. When reusing `chunk_kalman`, set its within-chunk decay input to $g = 0$ ($\alpha = 1$); the actual decay is applied only by equation (24). Passing tokenwise decay into that kernel would define a different mean estimator and require a different boundary-covariance derivation.

Let row j of O_c be $\mathbf{o}_j^\top$. Then

$$O_c = Q_c^\top S_c^- + \text{tril}(Q_c^\top L_c) E_c, \quad (18)$$

where `tril` includes the diagonal. Equivalently,

$$\mathbf{o}_j = (S_c^-)^\top \mathbf{q}_j + \sum_{t=1}^j (\mathbf{q}_j^\top \ell_t) \mathbf{e}_t. \quad (19)$$

Thus token j reads its own write and all earlier writes, but no later key or value contributes to its output. The full update in (17) is exposed only to the next chunk.

2.4 Dense Joseph covariance at the boundary

Although the gain is approximate, the covariance carried to the next chunk should describe the error of the estimator that was actually used. The final mean is linear in the boundary residual:

$$S_c^+ = S_c^- + L_{c,\text{eff}} \Delta_c, \quad L_{c,\text{eff}} = L_c \bar{A}_c. \quad (20)$$

Define

$$A_c = I - L_{c,\text{eff}} K_c^\top. \quad (21)$$

The dense boundary covariance is

$$\mathbf{P}_c^+ = \mathbf{A}_c \mathbf{P}_c^- \mathbf{A}_c^\top + \mathbf{L}_{c,\text{eff}} \mathbf{R}_c \mathbf{L}_{c,\text{eff}}^\top. \quad (22)$$

This is positive semidefinite in exact arithmetic. More precisely, it is the ensemble estimation-error MSE conditional on the chunk-entry variables, $\mathbf{K}_c$, and $\mathbf{R}_c$, but averaged over the current prior error and observation noise. It is not the Bayesian posterior covariance conditional on the realized $\mathbf{V}_c$. Propagating it gives average calibration for the chosen linear estimator; it does not erase that estimator’s realized conditional bias.

Equation (22) must not be replaced by the exact-posterior downdate $\mathbf{P}_c^- - \mathbf{U}_c \mathbf{G}_c^{-1} \mathbf{U}_c^\top$, because that covariance would correspond to a different estimator.

Equation (22) has the low-rank expansion

$$\boxed{\mathbf{P}_c^+ = \mathbf{P}_c^- - \mathbf{L}_{c,\text{eff}} \mathbf{U}_c^\top - \mathbf{U}_c \mathbf{L}_{c,\text{eff}}^\top + \mathbf{L}_{c,\text{eff}} \mathbf{G}_c \mathbf{L}_{c,\text{eff}}^\top} \quad (23)$$

and therefore needs only GEMM-shaped products with width C_c . The full $\mathbf{G}_c = \mathbf{R}_c + \mathbf{K}_c^\top \mathbf{U}_c$ is used in the covariance accounting, but it is never inverted or factored. Its off-diagonal entries correct the confidence carried forward even though they were omitted from the within-chunk gain.

Repeated or collinear keys expose why this distinction matters. Sequential residuals prevent the undamped parallel update from multiplying one identical residual by C_c , but the frozen gains still fail to contract after earlier observations. The estimator can therefore overweight later noise or become order-sensitive. The Joseph term reports the MSE of that behavior rather than pretending that the update was the exact block posterior.

2.5 Dynamics only at chunk boundaries

Chunk-KLA uses an update-then-predict convention. After $(\mathbf{S}_c^+, \mathbf{P}_c^+)$ is committed, apply one diagonal transition:

$$\begin{aligned} \mathbf{S}_{c+1}^- &= \mathbf{D}_c^{\text{chunk}} \mathbf{S}_c^+, \\ \mathbf{P}_{c+1}^- &= \mathbf{D}_c^{\text{chunk}} \mathbf{P}_c^+ (\mathbf{D}_c^{\text{chunk}})^\top + \mathbf{\Omega}_c^{\text{chunk}}. \end{aligned} \quad (24)$$

The first prototype reuses the existing token gates by aggregating their open-loop effect:

$$\mathbf{a}_c = \bigodot_{t=1}^{C_c} \boldsymbol{\alpha}_t, \quad \mathbf{D}_c^{\text{chunk}} = \text{diag}(\mathbf{a}_c), \quad (25)$$

$$\boldsymbol{\omega}_c^{\text{chunk}} = \sum_{s=1}^{C_c} \left(\bigodot_{u=s+1}^{C_c} \boldsymbol{\alpha}_u^2 \right) \odot \boldsymbol{\omega}_s, \quad \mathbf{\Omega}_c^{\text{chunk}} = \text{diag}(\boldsymbol{\omega}_c^{\text{chunk}}). \quad (26)$$

These gates affect only the next chunk, so using all positions of the current chunk does not leak future information into a current output. They preserve the cumulative open-loop decay and process noise across one chunk interval, but they do not reproduce the original order of C_c alternating predict–update steps. In particular, the full chunk decay attenuates a final-position write by the same amount as a first-position write, whereas tokenwise KLA applies only the remaining tail decay. For $C = 1$, the gate is also shifted from predict-before-update to update-before-predict. The quality screen must therefore compare update–predict, a causal pre-chunk gate, and an explicitly position-transported write before the gate convention is frozen. A learned chunk-level gate is a later modeling ablation, not part of the first kernel.

2.6 Initialization, tails, and complexity

Use $\mathbf{S}_0^- = \mathbf{0}$ and $\mathbf{P}_0^- = \mu^{-1}\mathbf{I}$. A final partial chunk uses its true length C_c ; padding must be masked from $\mathbf{K}, \mathbf{Q}, \mathbf{V}, \mathbf{R}$, the triangular output, and gate aggregation rather than represented as a finite-noise observation.

For one full chunk, the leading dense work is

$$O(d^2C + dC^2 + dCm + C^2m + C^3), \quad (27)$$

with recurrence depth T/C . The $O(Td^2)$ covariance-key arithmetic has not disappeared, but it becomes tensor-core-friendly matrix multiplication. The exact dynamic builder, factorization, and wide triangular solve are absent; the C^3 term is the existing unit-lower WY transform. With the production geometry $d = m = 128$, $C = 32$ is the provisional first point because exact component evidence favors that tile and its local matrices remain small. The new dependent recurrence must select among $C \in \{16, 32, 64\}$; the old exact-component optimum is not automatically the new optimum. The batching benefit primarily targets training and prefill. Autoregressive decode still needs a dense $\mathbf{P}\mathbf{k}_t$ product per token and can have a boundary Joseph-update latency spike.

3 What Is Exact and What Is Approximate

3.1 Exact properties of the proposed estimator

For the stated chunk-static observation model and the chosen gain $\mathbf{L}_c$:

1. Equations (15) and (18) are exactly the frozen-gain sequential-residual update and all of its causal prefixes.
2. Equation (22) is exactly the pre-observation ensemble MSE covariance of equation (17), conditional on the entry variables, keys, and noises variances.
3. The dense covariance preserves cross-key-coordinate uncertainty between chunks.
4. With $C_c = 1$, the observation update is the ordinary scalar Kalman update. Full parity with equation (1) additionally requires the same placement of the diagonal prediction.

3.2 Deliberate approximations

The model changes tokenwise ExactKLA in three places:

1. Every raw gain is computed from the same boundary covariance, so it ignores covariance contraction from earlier within-chunk observations.
2. The resulting effective block gain is the causal WY transform in equation (20), not the exact joint gain $\mathbf{U}_c \mathbf{G}_c^{-1}$.
3. The latent state is held fixed during the chunk and dynamics are applied only at the boundary.

The approximation should be easiest when within-chunk keys are weakly correlated or observation noise is large. Repeated keys, nearly collinear keys, small r_t , abrupt dynamics, and tasks that need accurate uncertainty contraction are the required stress cases. Larger chunks improve parallelism but increase all three approximation errors.

3.3 Two bounded alternatives

The first alternative removes the WY dependency by taking a damped parallel Jacobi step

$$\mathbf{S}_c^+ = \mathbf{S}_c^- + \eta_C \mathbf{L}_c \mathbf{\Delta}_c. \quad (28)$$

It is only a speed/quality ablation. The undamped choice $\eta_C = 1$ is unsafe: with identical scalar keys, negligible noise, and identical targets, its error multiplier is $1 - C$. At $C = 32$ it can amplify the error by $31\times$ in one chunk. Any damping must be fixed from causal-safe information such as C ; a rule based on future keys cannot affect earlier outputs. Use the corresponding effective gain $\eta_C \mathbf{L}_c$ in Joseph accounting.

The second fallback is for a speed failure dominated by dense PK work. Compress the chunk to rank $r \ll C$ using a key- or position-dependent matrix $\mathbf{W}_c \in \mathbb{R}^{C \times r}$ with $\text{rank}(\mathbf{W}_c) = r$. The projection may depend on conditioned keys and positions, but not on realized values or observation noise:

$$\widetilde{\mathbf{K}}_c = \mathbf{K}_c \mathbf{W}_c, \quad \widetilde{\mathbf{V}}_c = \mathbf{W}_c^\top \mathbf{V}_c, \quad \widetilde{\mathbf{R}}_c = \mathbf{W}_c^\top \mathbf{R}_c \mathbf{W}_c. \quad (29)$$

Then

$$\widetilde{\mathbf{G}}_c = \widetilde{\mathbf{R}}_c + \widetilde{\mathbf{K}}_c^\top \mathbf{P}_c^- \widetilde{\mathbf{K}}_c, \quad \widetilde{\mathbf{L}}_c = \mathbf{P}_c^- \widetilde{\mathbf{K}}_c \widetilde{\mathbf{G}}_c^{-1}. \quad (30)$$

One rank- r block Kalman update then commits the boundary memory at cost $O(d^2r + dr^2 + r^3 + drm + r^2m)$, plus summary construction. The shorter $O(d^2r + dr^2 + r^3)$ expression applies when $m = d$ and only leading dense terms are counted. The projection must be matched across key, value, and noise; pooling keys and values independently is not a coherent observation model. Because the summary is available only at the boundary, this variant needs a separate local causal path for current-chunk outputs. Evaluate $r \in \{1, 4, 8\}$ only after the full- C model establishes the quality ceiling.

4 Kernel Architecture

4.1 Forward dataflow

The Stage-5 feasibility prototype is a multi-launch CUDA/PyTorch decomposition at provisional $C = 32$. For each batch-head item it performs:

1. Load the dense boundary $\mathbf{S}_c^-$ and $\mathbf{P}_c^-$.
2. Compute $\mathbf{U}_c = \mathbf{P}_c^- \mathbf{K}_c$ with a GEMM-shaped operation.
3. Compute $\mathbf{G}_c = \mathbf{R}_c + \mathbf{K}_c^\top \mathbf{U}_c$, extract its diagonal, and scale columns to obtain $\mathbf{L}_c$.
4. Build the strictly lower $\mathbf{M}_c$, apply the existing WY transform to obtain $\overline{\mathbf{A}}_c$ and $\mathbf{E}_c$, and compute the causal outputs.
5. Form $\mathbf{L}_{c,\text{eff}} = \mathbf{L}_c \overline{\mathbf{A}}_c$, commit $\mathbf{S}_c^+$, and evaluate equation (23) with low-rank GEMMs.
6. Apply equation (24), retain the new dense boundaries, and advance to the next chunk.

The exact $C = 32$ work showed that one launch per chunk and materialized intermediate systems are not viable. The intended final schedule is therefore one persistent launch per sequence region, with one or two cooperative CTAs per batch-head item. The existing two-CTA evidence informs tile ownership and synchronization, but the new kernel contains no factor/inverse or 288-RHS path.

Grouped library GEMMs are allowed only in the Stage-5 multi-launch roofline decomposition. They cannot be called from a persistent device kernel. Stage 6 must use in-kernel MMA/CuTe/Triton dot operations, or explicitly remain a multi-kernel graph with materialized boundaries.

At $d = m = 128$, FP32 $\mathbf{P} + \mathbf{S}$ already require 128 KiB per head before $\mathbf{K}, \mathbf{Q}, \mathbf{V}, \mathbf{U}, \mathbf{L}, \mathbf{E}$, and small matrices. Before implementing the complete persistent body, freeze row/tile ownership and compile a representative spine. Report registers, static and dynamic shared memory, stack/local/spill/global scratch, active clusters for batch-head 64, and a synchronization/state-publication-only $T = 4096$ control. Compare one CTA and two CTAs in the same run; the prepared exact solver does not establish occupancy for this larger state.

4.2 Numerical policy

Store and accumulate $\mathbf{P}, \mathbf{G}$, gain denominators, the WY transform, and the Joseph update in FP32 initially. IEEE FP32, TF32, TF32x3/B3, and BF16-input GEMMs are separate measured arms; no operation is called tensor-core-safe before a full boundary/output/gradient gate promotes its instruction precision. Tests must be invariant to the ambient PyTorch matmul policy. Symmetrize the stored boundary covariance as $\frac{1}{2}(\mathbf{P} + \mathbf{P}^\top)$ to remove antisymmetric roundoff. Do not silently add jitter, clip eigenvalues, or project to the positive semidefinite cone. If stress tests require such a modification, record it as a model/numerical change and evaluate it explicitly.

The expanded Joseph expression can suffer cancellation even though the mathematical result is positive semidefinite. Compare it with direct Joseph evaluation in FP64 and with a square-root reference. The production path advances only if symmetry error and minimum-eigenvalue drift are no worse than the frozen FP32 baseline over full training-length sequences.

4.3 Backward and checkpointing

Automatic differentiation through the tensorized PyTorch model is the first gradient oracle. A custom backward recomputes $\mathbf{U}, \mathbf{G}, \mathbf{L}, \bar{\mathbf{A}}, \mathbf{E}, \mathbf{L}_{\text{eff}}$, output tiles, and Joseph terms inside each chunk. Storing every dense $(\mathbf{S}, \mathbf{P})$ boundary is too expensive: at $B = 4, H = 16, T = 4096, C = 32, d = m = 128$, it is approximately 1 GiB per layer before other activations. Use two-level checkpointing:

1. store one boundary pair every J chunks;
2. during backward, replay the J -chunk superchunk once and retain only its local boundaries;
3. reverse those local chunks, release them, and move to the preceding superchunk.

Sweep $J \in \{4, 8, 16\}$ and report the memory-recompute curve in a full 16-layer forward-backward run, including peak allocated, reserved, and external memory. No per-token dense covariance is ever stored.

5 Replacement Implementation Plan

5.1 Stage 0: Freeze the decision and baselines

Create `sanboxes/kernels/chunk_kla_dense_boundary_v0/`. Production `lit_gpt/` files remain unchanged through the correctness, quality, and speed screens. Pin the following same-process baselines at $B = 4, T = 4096, D = 2048, H = 16, d = m = 128$:

1. KDA and production DiagKLA;

2. the runnable exact direct-conditioning layer;
3. the best available $C = 32$ exact component, labeled explicitly as a component rather than a layer.

The fused recurrent ExactKLA backend is $d = 64$ -only, so use it at reduced shapes for correctness and approximation curves, not as a production $d = 128$ timing baseline. Record forward, forward-backward, peak memory, launch count, and allocator traffic. Use a source-pinned, same-process balanced A/B schedule (prefer the existing eight-position Latin cycle) with the same stream, warmup, and input tensors. Report both $\text{median}(\text{Chunk})/\text{median}(\text{KDA})$ and $\text{min}(\text{Chunk})/\text{min}(\text{KDA})$.

5.2 Stage 1: Independent FP64 references

Implement two independent references:

1. a token-loop reference that forms each frozen-prior gain, residual, causal prefix output, mean commit, Joseph covariance, and chunk transition directly;
2. a tensorized FP64 reference that explicitly forms the unit-lower WY inverse and implements equations (7)–(24).

The two references must agree, but neither is required to equal tokenwise ExactKLA for $C > 1$. Compare to ExactKLA only as a measured approximation gap. The $C = 1$, no-dynamics case must match one scalar ExactKLA measurement update to FP64 tolerance.

5.3 Stage 2: Semantic and numerical gates

Before any custom kernel, test:

1. random dense positive-definite and highly anisotropic boundary covariances;
2. $C \in \{1, 2, 4, 16, 32, 64\}$, partial tails, and multiple batch-head layouts;
3. tiny and large r , repeated and nearly collinear keys, zero keys, and large residuals;
4. decay gates near zero and one and small/large process noise;
5. equality of direct and expanded Joseph forms, covariance symmetry, and PSD behavior;
6. a Monte Carlo check that empirical estimation error matches the Joseph covariance for the stated static chunk model;
7. an op-level causality mutation test: changing any $(\mathbf{k}_t, \mathbf{v}_t, \mathbf{q}_t, r_t, \boldsymbol{\alpha}_t, \boldsymbol{\omega}_t)$ with $t > j$ cannot change output j ;
8. full-layer streaming-versus-full-sequence causality parity, including projections and short convolution;
9. chunk-boundary shifts, random offsets, and aligned versus layer-staggered boundaries;
10. a binding injected at each within-chunk position and queried in the next chunk for $\alpha \in \{0.5, 0.9, 0.99\}$, to expose the boundary-dynamics phase bias;
11. undamped Jacobi’s repeated-key failure, plus finite/norm-growth checks for frozen-gain sequential residuals and the $\eta_C = 1/C$ Jacobi ablation through $T = 4096$.

The future-key mutation may change the covariance committed for the next chunk; it must not change an earlier output in the current chunk.

5.4 Stage 3: Gradient gates

Run FP64 `gradcheck` on small cases and independent direct comparisons on larger cases. Check gradients separately with respect to $\mathbf{q}, \mathbf{k}, \mathbf{v}, r, \boldsymbol{\alpha}, \boldsymbol{\omega}, \mu, \mathbf{S}_0$, and a general dense $\mathbf{P}_0$. Parameterize the latter through a Cholesky factor or symmetric perturbation so `gradcheck` stays inside the SPD contract. Include losses on current outputs, final mean, and final covariance so no boundary adjoint is accidentally omitted. Repeat the repeated-key and small-noise stresses because they are the hardest cases for the approximate gain and Joseph cancellation.

5.5 Stage 4: Early quality screen

Run the tensorized reference, before writing a persistent kernel, on:

1. MQAR with repeated and colliding keys;
2. a short-range task that exposes within-chunk adaptation;
3. NIAH or an equivalent long-range retrieval probe;
4. a short language-model training smoke test.

Compare frozen-gain sequential residuals and damped Jacobi at $C \in \{4, 16, 32, 64\}$ against ExactKLA, DiagKLA, and KDA at matched parameter count. Report fixed and randomized offsets and the boundary-position survival probe. Before running this stage, freeze a manifest of numerical go/no-go thresholds for repeated-key state/covariance growth, MQAR and collision accuracy, short-range loss, NIAH accuracy, and a fixed-token language-model loss/perplexity gap. Derive thresholds from baseline seed variance; qualitative labels are not pass criteria. Select the update semantics and chunk width before Stage 5.

5.6 Stage 5: Tensorized GPU feasibility

Build a batched PyTorch/CUDA decomposition with no Cholesky, LDL^T , or exact-innovation solve. Reuse or adapt the unit-lower WY transform from `chunk_kalman`. Profile the complete recurrence and these regions:

1. dense $\mathbf{PK}$;
2. $\mathbf{K}^T \mathbf{U}$, normalization, and $\mathbf{L}$;
3. WY transform, residual, and causal output;
4. mean and Joseph boundary commits;
5. chunk dynamics and boundary traffic.

Benchmark $C = 16, 32, 64$, beginning with provisional $C = 32$. The timed call must be one literal $T = 4096$ recurrence in which chunk $c + 1$ consumes the actual $(\mathbf{S}, \mathbf{P})$ emitted by chunk c . Add poisoned-boundary and transitive-mutation checks; independently prepared systems, repeated buffers, and sums of component minima are not recurrence evidence.

After a fresh KDA measurement, freeze numeric budgets for the guards-off dependent core, mandatory tail/symmetrization work, layer peripherals, forward-backward, and peak memory. A component timing is never promoted to a layer claim. The complete-layer target for this approximate model is below $3\times$ KDA at matched $d = 128$; the old $5\text{--}6\times$ allowance applied to a model that

preserved exact inference. A 3–5 $\times$ approximate result advances only under a predeclared quality exception, and a result at or above 5 $\times$ is rejected. This stricter target is the required return for giving up exact tokenwise semantics.

5.7 Stage 6: Persistent forward and custom backward

Implement the persistent forward at the width selected in Stages 4–5. Start with one CTA per batch–head item, then test the measured two-CTA ownership pattern for the PK , output, and Joseph tiles. Remove global materialization of $U, G, L, \bar{A}, E, L_{\text{eff}}$, and Δ across kernel boundaries. Use current-stream execution and test arbitrary positive batch–head counts, 64-bit offsets, redzones, active-range aliases, allocator lifetime, and producer/consumer ordering.

After the forward passes all gates, implement the reverse chunk recurrence with superchunk checkpointing. Measure $J \in \{4, 8, 16\}$, validate every input gradient independently, and report forward–backward latency and peak memory. Freeze a separate numeric forward–backward/KDA ratio; forward speed alone cannot qualify a training kernel. The final timed layer call includes masks, tails, symmetrization, boundary publication, mandatory dispatches, and the complete layer shell.

5.8 Stage 7: Integration and ablation

Only after correctness, quality, forward–backward speed, and memory gates pass:

1. add a new op under `lit_gpt/kla_ops/`;
2. add a separate layer and `mixer_type`, never an `exact_kla` mode alias;
3. preserve explicit eager/reference and persistent modes;
4. run full-layer parity, a multi-step optimizer smoke test, and a short resumed training job;
5. ablate C , boundary offset, Joseph versus any approved numerical variant, and chunk-gate parameterization;
6. define the streaming cache and measure per-token Pk_t , boundary-update spikes, reset/tail behavior, and p50/p99 decode latency before claiming decode speed;
7. only then evaluate the rank- r boundary-summary fallback.

6 Paths Explicitly Not in This Plan

The following have already been explored or contradict the new model and must not silently reappear:

1. exact tokenwise dense Riccati recurrence inside each chunk;
2. exact prefix conditioning with the dynamic cross-covariance $G/J/B$ builder;
3. batched vendor POTRF/TRSM or a custom Cholesky/ LDL^T solver;
4. the register-resident $C = 32$ factor/inverse and 288-RHS schedule;
5. using a full post-chunk memory to produce every current-chunk output, which is noncausal;
6. using the exact-posterior covariance downdate with the approximate mean gain;

7. claiming ExactKLA equivalence for $C > 1$.

If the approximation misses its quality target, the response is to reduce C , test the specified damped-Jacobi ablation, or evaluate a local path plus rank- r boundary summary. It is not to restore the failed exact chunk implementation under a new name.

A Derivation of the Boundary Covariance

For one value coordinate, suppress the chunk subscript and condition throughout on the chunk-entry variables, $\mathbf{K}$, and $\mathbf{R}$, but not on the current realized observation. Write the boundary estimation error as

$$\mathbf{e}^- = \mathbf{s}^* - \mathbf{s}^-, \quad \mathbb{E}[\mathbf{e}^-] = \mathbf{0}, \quad \mathbb{E}[\mathbf{e}^-(\mathbf{e}^-)^\top] = \mathbf{P}^-. \quad (31)$$

The residual vector is

$$\boldsymbol{\delta} = \mathbf{v} - \mathbf{K}^\top \mathbf{s}^- = \mathbf{K}^\top \mathbf{e}^- + \boldsymbol{\epsilon}, \quad \mathbb{E}[\boldsymbol{\epsilon}\boldsymbol{\epsilon}^\top] = \mathbf{R}. \quad (32)$$

For an arbitrary chosen block gain $\mathbf{L}$, the estimator and its error are

$$\mathbf{s}^+ = \mathbf{s}^- + \mathbf{L}\boldsymbol{\delta}, \quad (33)$$

$$\mathbf{e}^+ = (\mathbf{I} - \mathbf{L}\mathbf{K}^\top)\mathbf{e}^- - \mathbf{L}\boldsymbol{\epsilon}. \quad (34)$$

The prior error and observation noise are independent, so

$$\mathbf{P}^+ = (\mathbf{I} - \mathbf{L}\mathbf{K}^\top)\mathbf{P}^-(\mathbf{I} - \mathbf{L}\mathbf{K}^\top)^\top + \mathbf{L}\mathbf{R}\mathbf{L}^\top. \quad (35)$$

Let $\mathbf{U} = \mathbf{P}^-\mathbf{K}$ and $\mathbf{G} = \mathbf{R} + \mathbf{K}^\top\mathbf{U}$. Expanding gives

$$\mathbf{P}^+ = \mathbf{P}^- - \mathbf{L}\mathbf{K}^\top\mathbf{P}^- - \mathbf{P}^-\mathbf{K}\mathbf{L}^\top + \mathbf{L}(\mathbf{K}^\top\mathbf{P}^-\mathbf{K} + \mathbf{R})\mathbf{L}^\top \quad (36)$$

$$= \mathbf{P}^- - \mathbf{L}\mathbf{U}^\top - \mathbf{U}\mathbf{L}^\top + \mathbf{L}\mathbf{G}\mathbf{L}^\top, \quad (37)$$

which proves equation (23). The result holds for the proposed diagonal- innovation effective gain, the exact block gain, or any other gain. In the proposed model, substitute $\mathbf{L} = \mathbf{L}_{\text{eff}}$. The expectation averages over the current prior error and observation noise. Conditional on a realized observation, the approximate estimator generally has a nonzero error bias relative to the exact posterior mean.

B Relation to the Exact Block Posterior

For the chunk-static Gaussian model, the conditional mean minimizes

$$\frac{1}{2} \text{Tr}[(\mathbf{S} - \mathbf{S}^-)^\top (\mathbf{P}^-)^{-1} (\mathbf{S} - \mathbf{S}^-)] + \frac{1}{2} \left\| \mathbf{R}^{-1/2} (\mathbf{V} - \mathbf{K}^\top \mathbf{S}) \right\|_F^2. \quad (38)$$

Solving the normal equations yields

$$\mathbf{S}_{\text{block}}^+ = \mathbf{S}^- + \mathbf{P}^-\mathbf{K} \left(\mathbf{R} + \mathbf{K}^\top \mathbf{P}^-\mathbf{K} \right)^{-1} (\mathbf{V} - \mathbf{K}^\top \mathbf{S}^-). \quad (39)$$

Chunk-KLA first forms independent scalar gains and then applies the causal WY transform:

$$\mathbf{L}_{\text{eff}} = \mathbf{U}\mathbf{D}_g^{-1} \left[\mathbf{I} + \text{tril}(\mathbf{K}^\top \mathbf{U}\mathbf{D}_g^{-1}, -1) \right]^{-1}. \quad (40)$$

This generally differs from $\mathbf{U}\mathbf{G}^{-1}$, so the proposed mean is not the minimizer of the full block objective. It is the exact result of sequential residual updates with gains frozen at the boundary. Even when $\mathbf{G}$ is not diagonal, this causal transform can suppress repeated identical residuals; it does not reproduce the exact covariance contraction or the exact block posterior.

C Causality

Fix an output position j . The boundary variables $(\mathbf{S}_c^-, \mathbf{P}_c^-)$ depend only on preceding chunks. For every $t \leq j$, ℓ_t depends only on $\mathbf{P}_c^-$, $\mathbf{k}_t$, and r_t , while $\Delta_{c,t}$ depends only on $\mathbf{S}_c^-$, $\mathbf{k}_t$, and $\mathbf{v}_t$. The leading $j \times j$ block of the unit-lower $\bar{\mathbf{A}}_c$ depends only on positions at or before j . Equation (18) masks all terms $t > j$. Hence changing a future key, value, query, noise value, or gate cannot change $\mathbf{o}_j$.

The dense boundary covariance uses the entire chunk through $\mathbf{G}_c$, and the aggregated dynamics use the entire chunk through its gates. Both become visible only to chunk $c + 1$, after all of chunk c is in the causal past. This is why a dense post-chunk commit is causal even though substituting that same post-chunk state into earlier outputs would not be.